CIS5200: Machine Learning Spring 2026

7: Logistic Regression

Date: February 5, 2026

In linear regression, because the labels were real valued, it made sense to use a linear function $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ directly as a predictor. This is simply because $\mathbf{w}^\top \mathbf{x}$ in principle can map $\mathbf{x}$ to any real value, and the labels are real valued in the regression setting. In the classification setting, we'll need to consider models that ultimately output either $+1$ or -1: a "prediction" of some real number like -0.7 is guaranteed to be wrong when we know the labels are by definition either $+1$ or -1. Broadly speaking, there are two approaches to taking some model that outputs a real value and making it instead output either $+1$ or -1:

1. **Take the sign.** If we take $h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$, then $h(\mathbf{x})$ outputs $+1$ whenever $\mathbf{w}^\top \mathbf{x}$ is positive, and -1 when it's negative. Thus, our goal will be to learn a $\mathbf{w}$ so that $h(\mathbf{x})$ maps positive points to positive values and negative points to negative values. If you remember, this was the approach the perceptron took!

2. **Model the probability.** An alternative to directly taking the sign is to have $h(\mathbf{x})$ output an estimate of the **probability** that $\mathbf{x}$ has label $+1$, $\eta(\mathbf{x}) := \Pr[y = +1 \mid \mathbf{x}]$. Remember that for binary classification and 0/1 loss, the Bayes optimal classifier was exactly this quantity! So the better we can estimate this quantity, the closer we get to the Bayes optimal solution. If we have some function that outputs a real value, we can use a **squashing function** $\sigma(\cdot)$ to achieve this. A squashing function takes a real number as input and maps it to a value between 0 and 1: $\sigma : \mathbb{R} \to [0, 1]$. If $h(\mathbf{x})$ outputs the probability that $y = +1$, then of course $1 - h(\mathbf{x})$ is the probability that $y = -1$. Given such a model, we can simply choose the label with the highest probability.

1 Logistic Regression

Logistic regression starts with the latter of these two options. To begin with, we introduce the **sigmoid function** $\sigma : \mathbb{R} \to [0, 1]$ as an important squashing function:

$$\sigma(v) = \frac{1}{1 + \exp(-v)} \tag{1}$$

The sigmoid function takes a real value v and maps it to the range $[0, 1]$. To see this, think about what happens as $v \to \infty$, $v \to -\infty$, and $v = 0$.

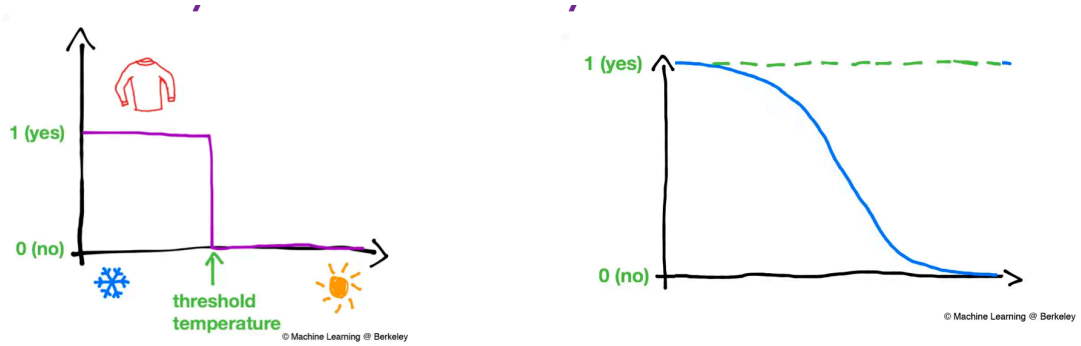

Figure 2: Decision boundaries for deciding on wearing a sweater based on outside temperature [source: `https://medium.com/@ml.at.berkeley/machine-learning-crash-course-part-2-3046b4a7f943`]

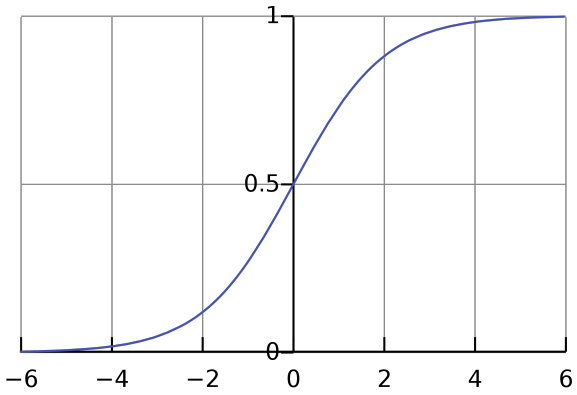

Figure 1: The function $\frac{1}{1+\exp(-v)}$ as a function of v.

Given a particular linear model parameterized by $\mathbf{w}$, we can map the real valued predictions of the model $\mathbf{w}^\top \mathbf{x}$–often called the **logits** in logistic regression–to a probability as follows:

$$h(\mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}$$

Why model the probability instead of taking the sign? One of the drawbacks of using the sign in Perceptron was the assumption of deterministic and noise-free labels. In real-life, our decisions are often non-deterministic and noisy. For instance, suppose you need to decide whether to wear a sweater when you leave house. You would check the temperature outside and decide based on that. It is often not a single temperature flip that governs this, often it depends on other factors such as how sunny or windy it is, or how you are feeling today. Thus, it is more likely that within a range you would sometimes wear a sweater and sometimes not, but your decision is more certain when it is either the temperature is too low or too high. Figure 2 captures this example.

Let us formalize this by denoting $\eta(\mathbf{x})$ as the conditional probability of label 1 given input $\mathbf{x}$ under

the (unknown) data distribution $\mathcal{P}$.

$$\eta(\mathbf{x}) = \Pr[y = 1 \mid \mathbf{x}].$$

In the case of the Perceptron setup, the labels were deterministic, therefore $\eta(\mathbf{x}) \in \{0, 1\}$. In general, $\eta(\mathbf{x}) \in [0, 1]$.

In the setting of logistic regression, we model η using a sigmoid function applied to a linear function, in particular,

$$\Pr[y = 1 \mid \mathbf{x}] = \sigma(\mathbf{w}^\top \mathbf{x}) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{x})}.$$

The sigmoid function can be thought of as a smooth approximation of the sign/step function, which allows for uncertainty near the boundary.

What does the boundary of logistic regression look like? With the sigmoid function around the linear model, it may not be clear what kind of decision boundary this model gives. However, even with a sigmoid, this turns out to be a linear model! Observe that we have,

$$\frac{\Pr[y = 1 \mid \mathbf{x}]}{\Pr[y = -1 \mid \mathbf{x}]} = \frac{\sigma(\mathbf{w}^\top \mathbf{x})}{1 - \sigma(\mathbf{w}^\top \mathbf{x})} = \exp(\mathbf{w}^\top \mathbf{x}).$$

When this ratio is 1, we are at the boundary, and when it is above 1, we predict 1 and -1 otherwise. Observe that this gives us a linear decision boundary as before.

Logistic loss. Having set up the model, now we are ready to talk about the loss function. The loss used in logistic regression is known as the logistic loss, and it has the following form,

$$\ell(f(\mathbf{x}), y) = \begin{cases} -\log(f(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - f(\mathbf{x})) & \text{otherwise} \end{cases}$$

For the particular function class we have chosen $\mathcal{H} := \{\mathbf{x} \mapsto \sigma(\mathbf{w}^\top \mathbf{x} + b) \mid \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$, this evaluates more compactly to

$$h(\mathbf{x}) := \sigma(\mathbf{w}^\top \mathbf{x}) \implies \ell(h(\mathbf{x}), y) = \log(1 + \exp(-y\mathbf{w}^\top \mathbf{x}))$$

This loss is an upper bound to the 0/1 loss we used for Perceptron. It penalizes incorrect prediction heavily as the points distance from the boundary increases. See Figure 3 for a plot of the losses. The figure also has hinge loss, $\max(0, 1 - y\ \mathbf{w}^\top \mathbf{x})$ which is also an upper bound on the 0/1 loss. These losses are known as *surrogate* losses which serve as an approximation to the 0/1 loss while being more amenable to optimization. Both logistic and hinge loss are convex.

Let's think for a moment about what this function does.

Suppose $y = +1$. Here, we are in the first case. $h(\mathbf{x})$ outputs our model's estimate of the probability that $y = +1$, so a number between 0 and 1. On the range $[0, 1]$ $log(v)$ is negative (going to $-\infty$ as $v \to 0$, and $log(1) = 0$). Thus, $-\log(v)$ is *positive* on the range $[0, 1]$: in fact, it grows to infinity as $v \to 0$ and approaches 0 as $v \to 1$. Putting this in the context of our machine learning model $h(\mathbf{x})$, this means that $-\log(h(\mathbf{x}))$ approaches 0 as $h(\mathbf{x})$ approaches 1, and it approaches ∞ as $h(\mathbf{x})$ approaches 0. In other words, we achieve 0 loss when $h(\mathbf{x})$ outputs 1–a good thing in the setting where $y = +1$!. Furthermore, as $h(\mathbf{x})$ approaches estimating $\Pr[y = +1 \mid \mathbf{x}] = 0$, the loss gets arbitrarily large. Again, a good thing when $y = +1$.

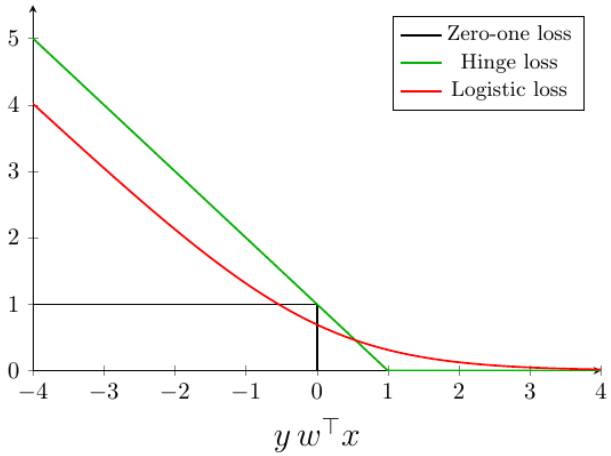

Figure 3: Comparison of 0/1 loss, logistic loss, and hinge loss as functions of the margin $y\,\mathbf{w}^\top\mathbf{x}$.

Now suppose $y = -1$. Here, we are in the second case, and we need to consider $-\log(1-h(\mathbf{x}))$. This has the exact opposite behavior: as $h(\mathbf{x})$ approaches 0, $1-h(\mathbf{x})$ approaches 1, and so $-\log(1-h(\mathbf{x}))$ approaches 0. This is a good thing when $y = 0$. Similarly, as $h(\mathbf{x})$ approaches 1, $1-h(\mathbf{x})$ approaches 0, and so $-\log(1-h(\mathbf{x}))$ approaches $-\log(0) = \infty$.

A more compact loss. The logistic loss is great in the sense that it does what we want: the loss approaches 0 for a correct classification, and grows arbitrarily large for a misclassification. But how did we get the more compact form? To see this, let's derive it in both cases.

Case 1: $y = +1$. Here,

$$-\log h(\mathbf{x}) = -\left[\log \frac{1}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\right] = -\Big[\log(1) - \log(1+\exp(-\mathbf{w}^\top\mathbf{x}))\Big] = \log(1+\exp(-\mathbf{w}^\top\mathbf{x}))$$

Case 2: $y = -1$. Here,

$$-\log(1-h(\mathbf{x})) = -\left[\log\left(1 - \frac{1}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\right)\right]$$

Using the identity $1 - \frac{1}{1+v} = \frac{v}{1+v}$:

$$-\left[\log\left(1 - \frac{1}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\right)\right] = -\left[\log\left(\frac{\exp(-\mathbf{w}^\top\mathbf{x})}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\right)\right]$$

Now, we note that:

$$\frac{\exp(-\mathbf{w}^\top\mathbf{x})}{1+\exp(-\mathbf{w}^\top\mathbf{x})} = \frac{\exp(-\mathbf{w}^\top\mathbf{x})}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\frac{\exp(\mathbf{w}^\top\mathbf{x})}{\exp(\mathbf{w}^\top\mathbf{x})} = \frac{1}{1+\exp(\mathbf{w}^\top\mathbf{x})}$$

Therefore:

$$-\left[\log\left(\frac{\exp(-\mathbf{w}^\top\mathbf{x})}{1+\exp(-\mathbf{w}^\top\mathbf{x})}\right)\right] = -\left[\log\left(\frac{1}{1+\exp(\mathbf{w}^\top\mathbf{x})}\right)\right] = \log(1+\exp(\mathbf{w}^\top\mathbf{x}))$$

Since $y = -1$:

$$\log(1 + \exp(\mathbf{w}^\top \mathbf{x})) = \log(1 + \exp(-y\mathbf{w}^\top \mathbf{x}))$$

Summary. The setup for logistic regression then looks like:

- Input space $\mathcal{X} = \mathbb{R}^d$
- Label space $\mathcal{Y} = \{-1, 1\}$
- Hypothesis class $\mathcal{H} := \{\mathbf{x} \mapsto \sigma(\mathbf{w}^\top \mathbf{x} + b) \mid \mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}\}$ where $\sigma(a) = \frac{1}{1+\exp(-a)}$.
- Loss function $\ell(f(\mathbf{x}), y) = \begin{cases} -\log(f(\mathbf{x})) & \text{if } y = 1 \\ -\log(1 - f(\mathbf{x})) & \text{otherwise} \end{cases}$.

1.1 Why logistic loss? A probabilistic perspective

There is a very good justification for using this loss, if we take a probabilistic view. An alternate view of solving a supervised learning task is to

- First pick an explicit model on the data distribution,
- Then find parameters such that they maximize the likelihood of seeing the training data S.

This approach is known as the Maximum Likelihood Approach (MLE). Suppose the parameters of the underlying model are θ, then the likelihood is

$$\mathcal{L}(\theta) = \mathcal{P}(S \mid \theta) = \prod_{i=1}^{n} \mathcal{P}(\mathbf{x}_i, y_i \mid \theta).$$

Here the last equality follows from the i.i.d. assumption on the training dataset.

We use $\mathcal{P}$ to denote probability in the case of discrete variables, and the density of the distribution at the input. This distinction is important, since the probability of any $\mathbf{x}$ in a continuous distribution will be 0.

Example. Consider the data distribution being coin tosses of a coin with probability of heads being θ. Here θ parametrizes the different distributions of heads and tails we may observe when tossing the coin. Suppose we observe $S = \{t_1, \ldots, t_n\}$, a sequence of outcomes of tosses, where

$t_i = 1$ implies heads and -1 implies tails. Then we have,

$$\begin{aligned}
\mathcal{L}(\theta) &= \mathcal{P}(S|\theta) \\
&= \prod_{i=1}^{n} \mathcal{P}(t_i|\theta) \\
&= \prod_{i=1}^{n} (\mathbb{1}[t_i = 1]\theta + \mathbb{1}[t_i = -1](1-\theta) \\
&= \prod_{i=1}^{n} \theta^{\mathbb{1}[t_i=1]}(1-\theta)^{\mathbb{1}[t_i=-1]} \\
&= \theta^{\sum_{i=1}^{n} \mathbb{1}[t_i=1]}(1-\theta)^{\sum_{i=1}^{n} \mathbb{1}[t_i=-1]}
\end{aligned}$$

If the number of heads is denoted by m_H, then we have,

$$\mathcal{L}(\theta) = \theta^{m_H}(1-\theta)^{n-m_H}$$

Let's take log to make this easier.

$$\log \mathcal{L}(\theta) = m_H \log \theta + (n - m_H) \log(1-\theta)$$

Now minimizing with respect to θ gives us

$$\frac{m_H}{\theta} - \frac{n - m_H}{1-\theta} = 0 \implies \theta = \frac{m_H}{n}.$$

So for an observed sequence $\{H, T, H, H\}$, the MLE estimate is 3/4.

Conditional Likelihood. Often if we do not have an underlying model on $\mathbf{x}$, but only have one on $y \mid \mathbf{x}$ as in logistic regression, we can use the *conditional* likelihood. This is equivalent to taking a uniform prior over all the $\mathbf{x}$, i.e. we assume $\mathcal{P}(\mathbf{x}|\theta)$ is a constant and so $\mathcal{P}(\mathbf{x}, y|\theta) \propto \mathcal{P}(y|\mathbf{x}, \theta)$ where $\propto$ denotes proportional to (i.e. scaled by a constant). This is a common baseline assumption (i.e. that no particular $\mathbf{x}$ is more likely than others) Then,

$$\mathcal{L}(\theta) = \prod_{i=1}^{n} \mathcal{P}(y_i \mid \mathbf{x}_i, \theta).$$

Since we are concerned with maximizing the likelihood, a constant scale does not influence the final estimate. Let us compute this for our logistic model,

$$\begin{aligned}
\mathcal{L}(\mathbf{w}) &= \prod_{i=1}^{n} \mathcal{P}(y_i \mid \mathbf{x}_i, \mathbf{w}) \\
&= \prod_{i=1}^{n} \frac{1}{1 + \exp(-y_i \ \mathbf{w}^\top \mathbf{x}_i)}.
\end{aligned}$$

Again, we typically work with the log-likelihood as easier to work with, which is

$$\begin{aligned}
\log \mathcal{L}(\mathbf{w}) &= \log \left(\prod_{i=1}^{n} \frac{1}{1 + \exp(-y_i \ \mathbf{w}^\top \mathbf{x}_i)} \right) \\
&= -\sum_{i=1}^{n} \log(1 + \exp(-y_i \ \mathbf{w}^\top \mathbf{x}_i)).
\end{aligned}$$

Observe that this is just the negative of the logistic loss up to the scaling by n. So in this case, minimizing the logistic loss over the hypothesis class $\mathcal{H}$ is equivalent to maximizing the likelihood of the data under the probabilistic model given by the same hypothesis class $\mathcal{H}$. However, these are two different perspectives:

1. The former (loss minimization) seeks to find the model that makes the least "mistakes" where mistakes are defined by the loss function
2. The latter (probabilistic model) seeks to find a model that has the highest probability of having produced the observed data

The latter can be viewed as making less arbitrary "choices" as it does not need to pick a new loss function, and instead optimizes directly the likelihood of the data. It happens to be in this case that logistic loss corresponds to maximum likelihood, but this will not always be true for every loss function.

1.2 Putting things together

Now, suppose we are given a training dataset $\mathcal{D} : \{(\mathbf{x}_1, y_1), ..., (\mathbf{x}_n, y_n)\}$. The empirical risk on this dataset using the logistic loss becomes:

$$\hat{R}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} \log(1 + \exp(-y_i \mathbf{w}^\top \mathbf{x}_i)) \tag{2}$$

Here, unlike in linear regression, there is no closed form solution: we'll have to use gradient descent. In a homework assignment, you'll compute and implement the derivative of the above function with respect to $\mathbf{w}$. And that's it! We train logistic regression using gradient descent on the above objective, and then given a final $\mathbf{w}$, we make predictions of the probability that $y = 1$ for a point $\mathbf{x}$ via $h(\mathbf{x}) = \frac{1}{1+\exp(-\mathbf{w}^\top \mathbf{x})}$.